ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

Robotics & Computer Vision

Week 11: AI Line Following

Connecting Vision & AI to Robot Motion Control

Jet Racer • Turtlebot • Image → Error → Steering/Throttle

Lê Đình Phong, Ph.D

Week 11 in the Course Journey

1

Weeks 5–9

AI & Mapping — teaching robots to detect objects and locate lines

2

Week 10

Mobile Kinematics — understanding (v, ω), diff-drive, Ackermann models

3

Week 11

Vision/AI + Kinematics → autonomous line following on the track

4

Week 12

Visual Odometry — estimating trajectory from images

5

Week 13

Navigation & Obstacle Avoidance — full autonomous behavior

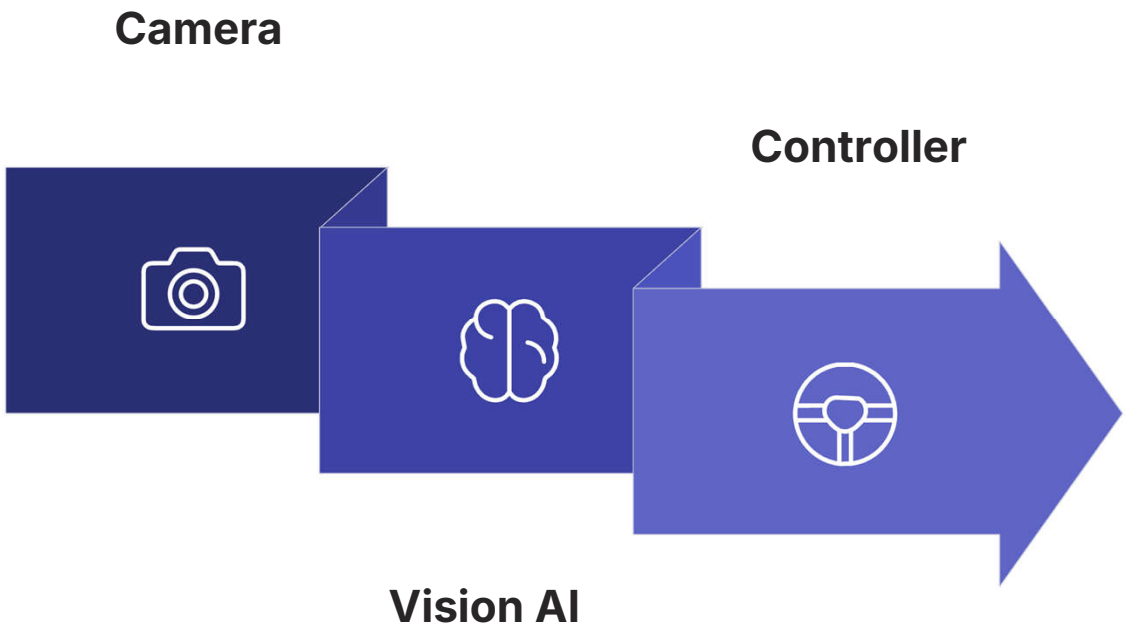
Learning Objectives — Week 11

1	2	3
Understand the Pipeline	**Design Control Laws**	**Hands-On Practice**
Camera image → processing/AI → error signal → control command	Implement P, PD, and PID controllers to compute steering, throttle, v, and ω from error	Simulate the system, then connect with real Jet Racer and Turtlebot hardware in the lab

What Is Line Following?

Given a line (typically black on white or white on black), the robot must **maintain its position on the line** while continuously moving forward. The sole information source is a downward-facing camera producing a continuous stream of images from the track.

Overall System Architecture



Each stage is modular — you can swap the vision method or the controller independently, making the system easy to test and improve.

Two Approaches to Extract Error

Classical Image Processing

Threshold → find centroid/edge → compute lateral offset from ROI center. Fast, interpretable, no training required.

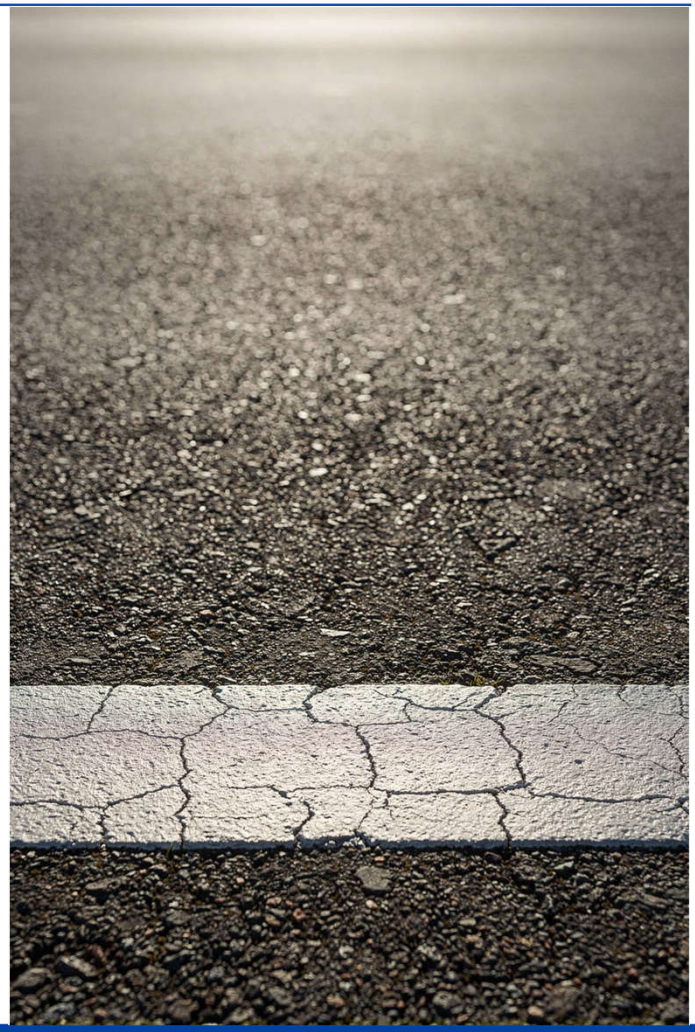
AI Regression (JetRacer Style)

A neural network directly predicts the target point (x, y) on the image representing the ideal driving path. Flexible for complex roads.

Both methods ultimately produce a **single real number** representing the error or target, feeding the same downstream controller.

Choosing the Region of Interest (ROI)

Only process the **lower strip** of the frame where the line is most likely to appear. This reduces noise from distant objects and people, and dramatically lowers computation load — critical for Jetson Nano or laptop deployments.



Classical Pipeline — Step by Step

01

Grayscale Conversion

Reduce 3-channel color image to single intensity channel

02

Gaussian Blur

Smooth the image to suppress pixel-level noise

03

Thresholding

Apply binary or Otsu threshold to isolate the line from the background

04

Contour / Centroid

Find contours or the binary mask of the line and compute its centroid x_c

Computing the Error Signal

Error Formula

Given ROI width **W_px** and line centroid **x_c**:
Image center: $x_o = W\_px / 2$
Normalized error: $e = (x\_c - x_o) / (W\_px / 2)$
This keeps $e \in [-1, 1]$ — negative means line is left of center, positive means right.

Intuition

e = 0 → robot is perfectly centered
e = −1 → line is at far left
e = +1 → line is at far right

OpenCV Code — Error Computation

```
def line_error_from_frame(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape
    roi  = gray[int(h * 0.6):h, :]       # lower 40% of frame

    _, bw = cv2.threshold(
        roi, 0, 255,
        cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
    )

    M = cv2.moments(bw)
    if M['m00'] == 0:
        return None                       # line lost!

    cx    = int(M['m10'] / M['m00'])
    error = (cx - w / 2) / (w / 2)        # normalize to [-1, 1]
    return error
```

Otsu thresholding automatically adapts the threshold to the image histogram — more robust than a fixed value.

AI Regression — JetRacer Road Following

NVIDIA JetRacer trains a regression CNN to predict a **target point (x, y)** on the image representing the best path ahead. Labels are collected by clicking the desired driving point on live camera images in a Jupyter notebook — no segmentation mask needed.

From x_target to Control Error

Network Output → Error

If the network outputs **x_target ∈ [0, W_px]**, normalize it the same way as the classical centroid:

$$e = \frac{x_{\text{target}} - W_{px}/2}{W_{px}/2}$$

Same scale as classical error → the same controller works for both methods.

Why This Is Powerful

The AI learns to anticipate curves earlier than centroid-based methods, giving smoother steering on complex tracks.

Proportional (P) Controller

Core Idea

If the line is to the left (e < 0), steer left. If to the right (e > 0), steer right. The correction is **proportional** to how far off-center the robot is.

$$\text{steering} = K_p \quad e, \quad K_p > 0$$

Tuning K_p

Too small → robot drifts off line slowly
Too large → robot oscillates wildly
Start around 0.5–1.0 and adjust empirically

PD / PID Controller — Smoother Tracking

P Term

Proportional response to current error. The main corrective force.

D Term

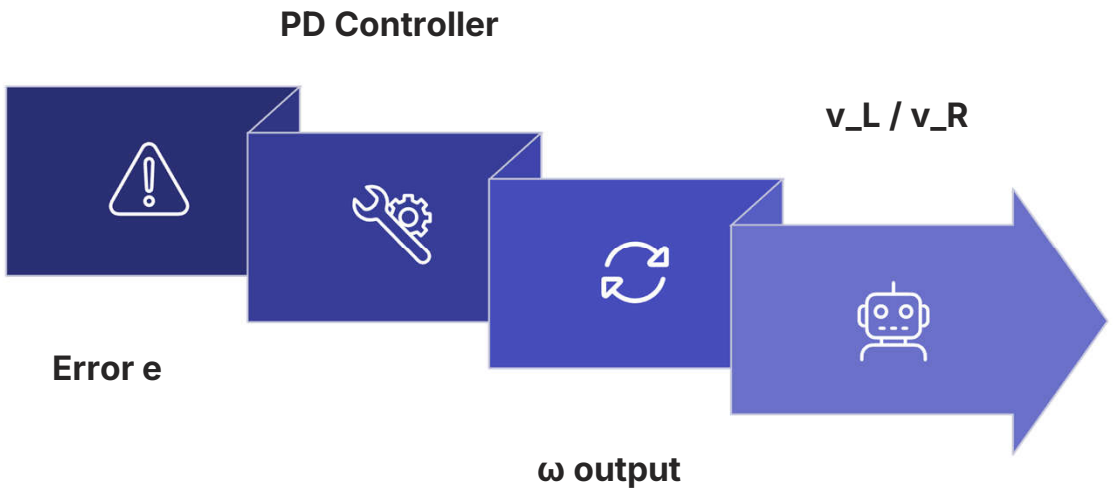
Uses **de/dt** to dampen overshoot and reduce oscillation. Critical for fast tracks.

I Term

Eliminates steady-state offset. Rarely needed in high-speed line following — can cause windup issues.

JetRacer AI Kit tutorials recommend a **PD controller** for steering as a practical sweet spot.

Controller → Turtlebot (Diff-Drive)



Equations

Set a constant forward speed: **v = v_const**
Compute angular velocity: **ω = K_p · e** (or PD/PID)
Then apply Week 10 kinematics:
v_L = v − ω·L/2
v_R = v + ω·L/2

Controller → Jet Racer (Ackermann)

Steering Command

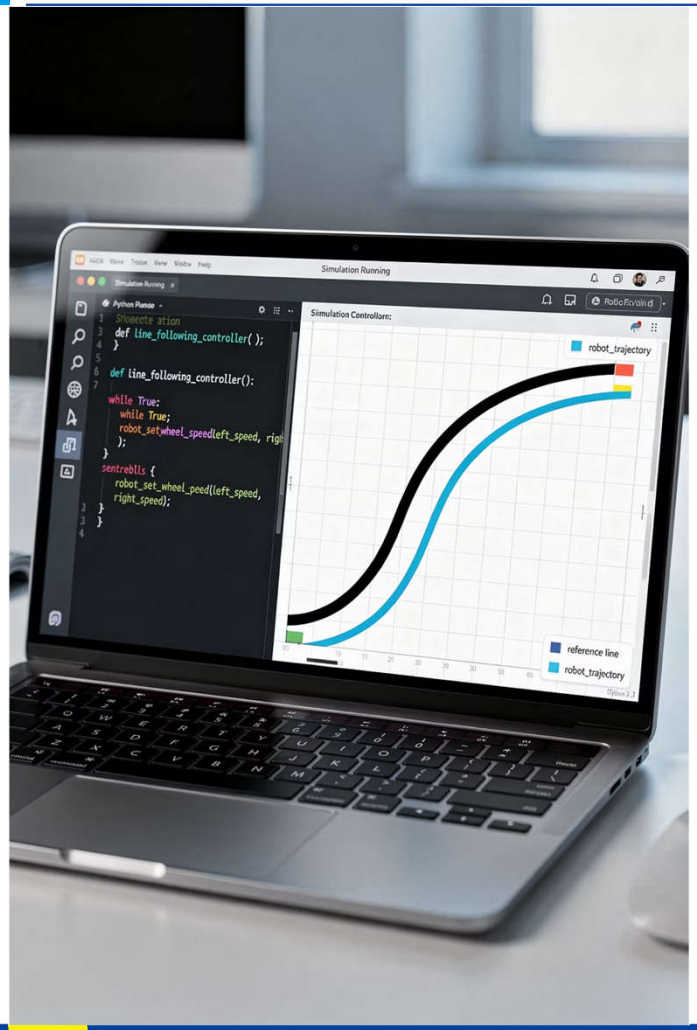
steering = K_s · e — maps normalized error to steering angle, clipped to [−1, 1] per JetRacer SDK.

Throttle Command

throttle = t_const — keep a fixed low speed during initial tuning to focus on steering behavior.

Output Clipping

Always apply **np.clip(steer, −1.0, 1.0)** to protect the servo and avoid mechanical damage.



Lab Overview — Week 11

Seven lab exercises, structured to build confidence step by step: start with **offline image analysis**, advance through simulation, then deploy on real hardware. Vision processing and control are tested independently before being combined.

LAB 11.1 → 11.7

LAB 11.1

Offline Image Processing for Line Following

Acquire Data

Record a video or image sequence of the track with visible line markings.

Build the Pipeline

Write an OpenCV script to display ROI, binary mask, centroid marker, and compute e frame by frame.

Analyze e(t)

Plot the error over time to visualize track difficulty — sharp spikes mean tight curves.

LAB 11.2

Unicycle Simulator + P Controller

Setup

Use the **unicycle_step** function from Week 10. Define a virtual line and compute lateral error **e** as the perpendicular distance from the robot to the line.

Experiment

Apply **ω = K_p · e**, **v = const**. Vary K_p and observe how the robot trajectory converges to (or oscillates around) the line. Plot (x, y) path.

LAB 11.3

Ackermann Simulator + Line Following

Use ackermann_step from Week 10

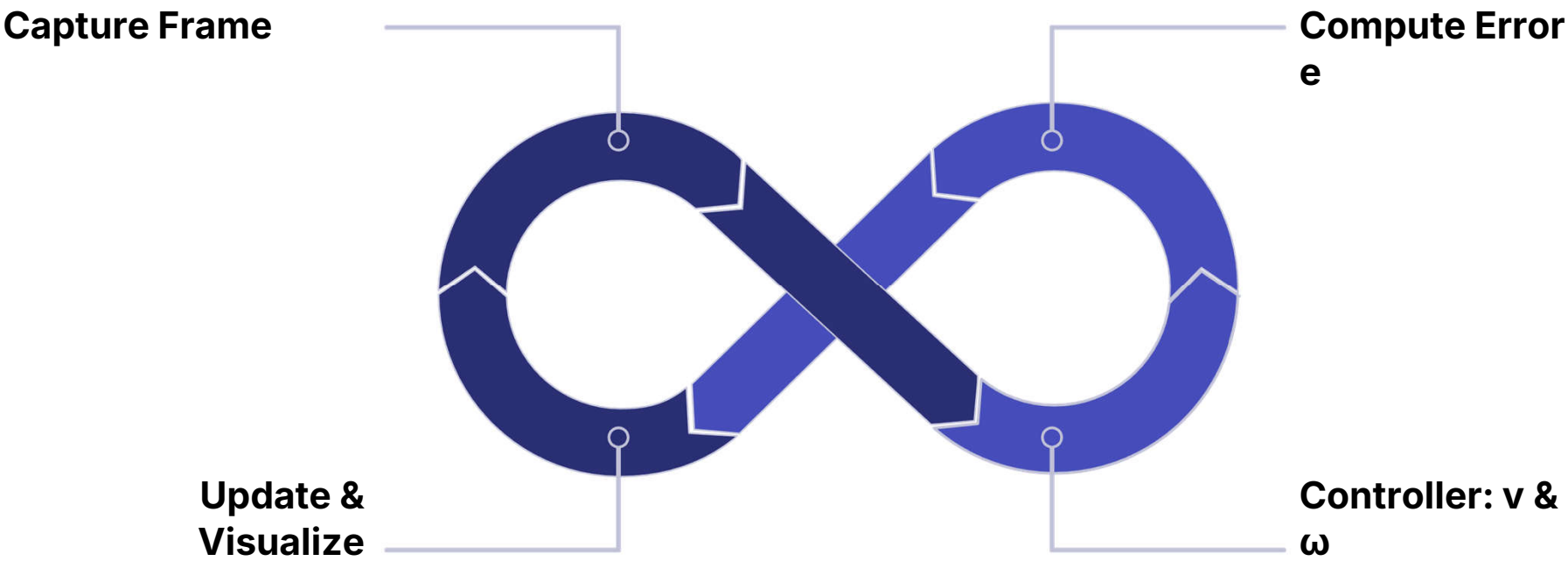
Feed recorded e(t) data into: **steering = K_s · e**, **throttle = t_const**. Simulate the Jet Racer's Ackermann kinematics.

Key Observation

Compare the trajectory shape with the unicycle model. Ackermann has a minimum turning radius — notice how it handles sharp curves differently.

LAB 11.4

Full Pipeline: Image + Controller in Simulation



Visualize the robot on the 2D plot or overlay its position on the camera image to verify end-to-end pipeline correctness before touching real hardware.

LAB 11.5

JetRacer Road Following — NVIDIA / Waveshare Tutorial

01

Open the Notebook

Launch **road_following.ipynb** from the jetracer or Waveshare JetRacer AI Kit repository.

02

Collect Data

Click the ideal driving point on 50–100 live images to label the target path.

03

Train the Model

Fine-tune a ResNet18 regression head. Training takes ~5 minutes on Jetson Nano.

04

Run the Demo

Deploy the model and watch the car follow the track autonomously in the lab.

LAB 11.6

Replace the JetRacer Controller

What to Do

Replace the default model-to-steering mapping in the NVIDIA notebook with your own **PD/PID controller**. Tune K_p and K_d manually.

What to Observe

Compare smooth, fast tracking (well-tuned PD) vs. oscillation (high K_p) vs. sluggish response (low gains). Record both runs for comparison.

LAB 11.7

Turtlebot Line Following (If Available)

Camera Node

Use the downward-facing camera; write a Python ROS node to compute error e from each frame.

Controller Node

Publish (v, ω) to **cmd_vel** using a P/PD law tuned for the Turtlebot's inertia.

Compare with Sim

Notice real-world effects: latency, inertia, floor reflections — all absent in simulation.

Real-World Challenges

Lighting Changes

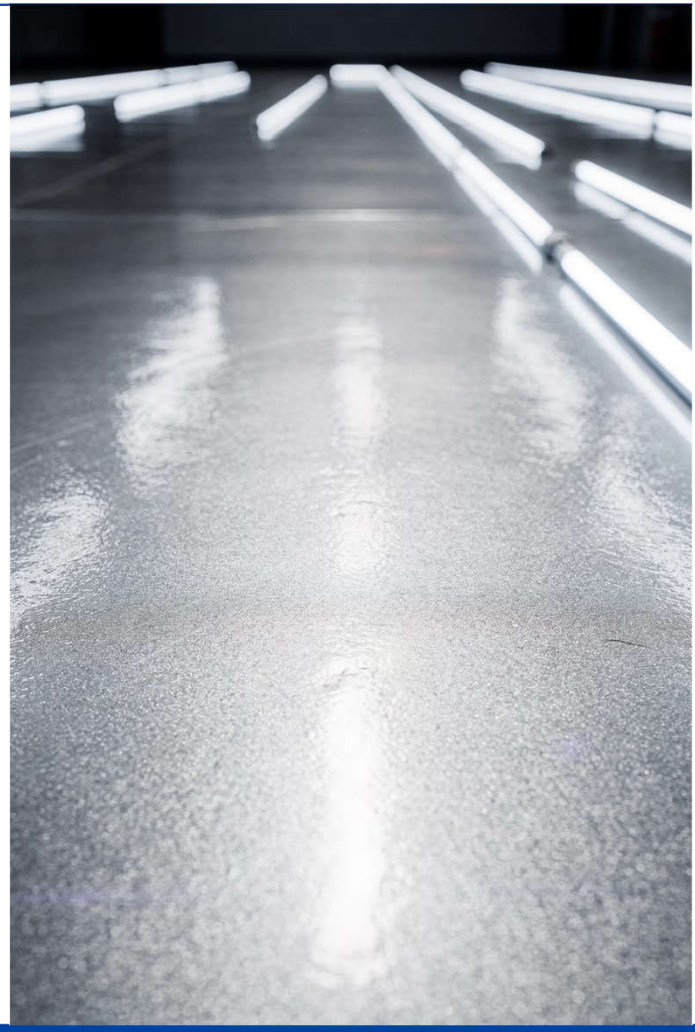
A fixed threshold fails under variable ambient light — shadows and reflections can make the line disappear or create false positives.

Line Imperfections

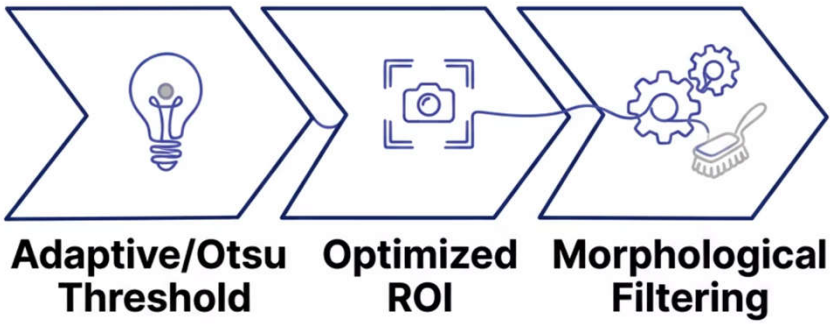
Faded tape, tape gaps, and shiny floor reflections all break simple threshold-based detection.

Processing Latency

Low frame rate at high speed means the robot acts on stale error data — stability degrades rapidly above a critical speed.



Robust Image Processing Techniques



Why Each Matters

Otsu thresholding automatically finds the optimal threshold from the image histogram — no manual tuning when lighting shifts.
ROI selection is the single most impactful preprocessing step for speed and noise reduction.
Morphology (dilate then erode) fills small gaps in a broken line and removes salt-and-pepper noise.

Speed Management & Steering Limits

Speed vs. Reaction Time

Higher speed leaves less time to react — reduce K_p or increase filtering aggressiveness as throttle increases.

Output Saturation

Always clip steering and throttle outputs to prevent motor burnout and avoid tipping the car on tight turns.

Error-Based Speed Reduction

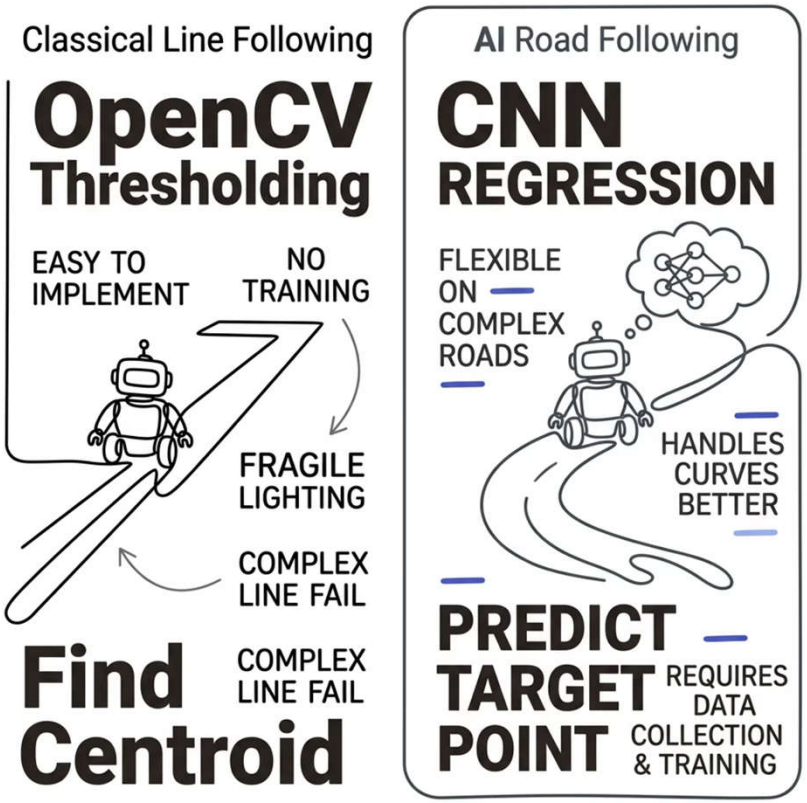
When $|e|$ is large, the robot is far off the line — automatically reduce speed so the controller has time to recover.

PD Controller Code — JetRacer (Waveshare Style)

```
prev_error = 0.0

while True:
    frame      = get_frame()
    e          = compute_error(frame)   # normalized to [-1, 1]

    de         = e - prev_error          # discrete derivative
    steer      = Kp * e + Kd * de
    steer      = np.clip(steer, -1.0, 1.0)

    car.steering = steer
    car.throttle = base_throttle

    prev_error = e
```

This mirrors the structure described in Waveshare JetRacer AI Kit tutorials. Start with **Kp ≈ 0.8, Kd ≈ 0.1** and tune from there.

AI Road Following vs. Classical Line Following



Which Should You Use?

The RCV course encourages you to try **both approaches**. Classical is the right starting point — it teaches the fundamentals. AI regression unlocks better performance on complex tracks and is the basis for real autonomous driving systems.

Performance Metrics for Line Following

e_{avg}

Mean Lateral Error

Average pixel or meter offset from line center over the run

σ^2

Error Variance

Oscillation measure — lower is smoother, higher means jitter

v_{max}

Max Stable Speed

Fastest speed where the robot still tracks without losing the line

N

Line-Loss Count

Number of times the robot leaves the track boundary during a run

Lab RCV Track Setup

Fixed Track

White tape on black surface — consistent, easy to threshold

Speed & Angle Limits

Restrict max steering and throttle during initial debug sessions

E-Stop Always Ready

Keep an emergency stop mechanism active on all Jet Racer and Turtlebot runs

Extensions: Multi-Lane, Intersections & Turns

Beyond Single Line

- Add lane priority logic when multiple lines are detected
- Detect intersections from line shape (T-junction, cross)
- Use AI segmentation to handle complex multi-lane environments

Scope Note

These topics go beyond the Week 11 core. They are excellent **final project** candidates — discuss with your instructor if you want to explore them.

Connection to Visual Odometry (Week 12)

Both line following and Visual Odometry (VO) use the **same downward-facing camera** and the same unicycle/Ackermann kinematics from Week 10.

Line Following

Uses images to **control** the robot — feedback to steer on the line

Visual Odometry

Uses images to **estimate** the robot's trajectory — open-loop pose reconstruction

Connection to Obstacle Avoidance (Week 13)

Week 11 — Follow the Line

The robot tracks an ideal reference path. The controller minimizes lateral error to the line.

Week 13 — Deviate When Needed

Obstacle detection from sensors/AI overrides the line-following command, blending path following with collision avoidance in the final controller.

Week 11 lays the control foundation that Week 13 will build on. Master error-based steering here before adding the complexity of obstacle avoidance.

Summary — Week 11 (Part 1)

Core Pipeline

Camera image → ROI → threshold/AI → centroid/target → normalized error $e \in [-1, 1]$ → steering / (v, ω) command

Two Vision Methods

Classical OpenCV (Otsu + centroid) for transparency; AI regression (JetRacer road following) for flexibility on complex tracks

Controllers

P controller is the simplest starting point; PD adds damping for smoother tracking; PID rarely needed for fast line following

Summary — Week 11 (Part 2)

Simulation First

Labs 11.1–11.4 build a solid sim foundation before touching real hardware

Real Hardware

Labs 11.5–11.7 deploy on Jet Racer and Turtlebot — real latency, inertia, lighting

Bridge to Future

Line following connects Week 10 kinematics to Week 12 VO and Week 13 navigation — a critical stepping stone

Q&A — AI Line Following

Open discussion topics:

Classical vs. AI

When would you choose OpenCV over a trained model, and vice versa?

Maximum Speed

What limits the top speed at which stable line following is achievable?

Common Demo Failures

What are the most frequent failure modes — and how would you diagnose them in real time?